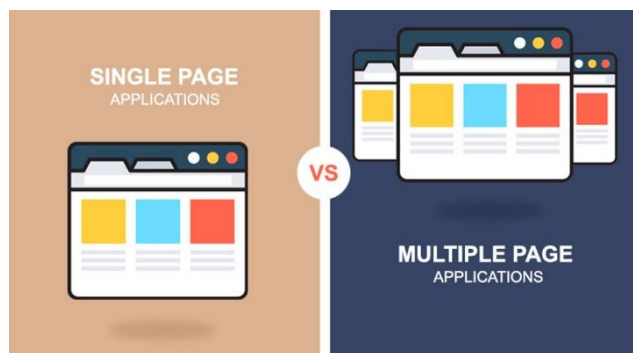


Тема 9. One page application (html, css, bootstrap). Frameworks принципи и стъпки за реализация.

I. Въведение	2
A. Определение на One Page Application (SPA).....	2
B. Защо да използваме SPA?	3
II. Архитектура на SPA	4
III. Използване на HTML, CSS и Bootstrap за разработка на интерфейса на SPA	6
A. HTML за структуриране на страницата	6
B. CSS за стилизиране и визуално оформление.....	6
C. Bootstrap за бързо и лесно оформление на дизайна.....	7
IV. Принципи на SPA Frameworks	8
V. Frameworks за SPA	9
1. React.....	10
2. Angular	10
3. Vue.js	10
4. Ember.js.....	11
5. Svelte	11
6. Aurelia	11
7. Backbone.js.....	12
8. Mithril	12
VI. Стъпки за реализация на SPA с използване на HTML, CSS и Bootstrap	13
A. Създаване на основната HTML структура.....	13
B. Добавяне на стилове с CSS.....	13
C. Използване на Bootstrap за създаване на респонсивен дизайн	13
D. Интегриране на фреймуърк за SPA.....	13
E. Създаване на компоненти и управление на състоянието.....	14
F. Добавяне на маршрутизация и навигация	14
VII. Примери за SPA	14
Програма за проследяване на разходите:	14
Проследяване на здравето	15
VIII. Предизвикателства на SPA.....	16

I. Въведение

Single-Page Applications (SPA) са навсякъде. Дори ако не сте съвсем сигурни какво са те, вероятно ги използвате редовно - те са отличен инструмент за създаване на невероятно ангажиращи и уникални преживявания за потребителите на уебсайтове.



SPA е уебсайт или уеб приложение, което “динамично” пренаписва текущата уеб страница с нови данни от уеб сървър, вместо стандартния метод на зареждане на цели нови страници от уеб браузър.

Лесно ще разпознаете някои популярни примери за SPA като Gmail, Google Maps, Airbnb, Netflix, Pinterest, PayPal и още много други. Компаниите по целия Интернет използват SPAs, за да създадат плътно, мащабируемо преживяване.



A. Определение на One Page Application (SPA)

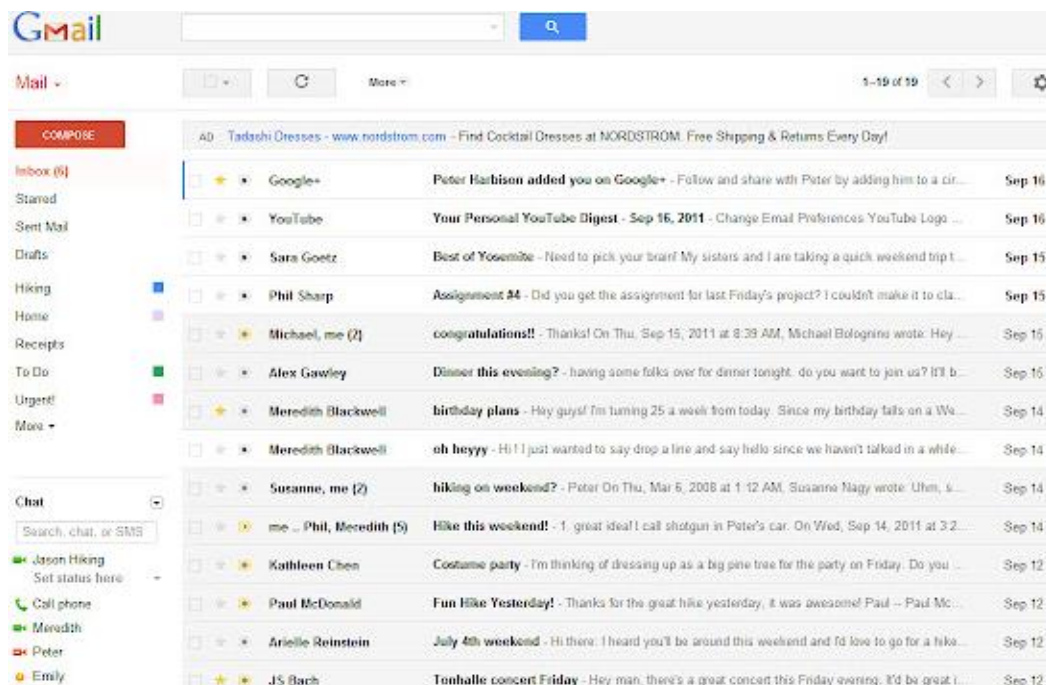
- ❖ **SPA е уебсайт или уеб приложение, което пренаписва текущата уеб страница с нови данни от уеб сървър, вместо стандартния метод на зареждане на цели нови страници от уеб браузър.**

SPA е страница (оттам и името), където голяма част от информацията остава същата, а само няколко елемента се актуализират едновременно.

Два са начините, по които *single-page applications* постигат пренаписването на съдържанието без презареждане. Или целият необходим код – *HTML, CSS и JS се зарежда директно* при зареждането на страницата, или *необходимите ресурси се зареждат динамично* и се добавят към страницата при необходимост, обикновено при дадено потребителско действие.

Например, когато преглеждате имейлите си, ще забележите, че по време на навигацията не се променя много — страничната лента и заглавната част остават непроменени, докато преглеждате входящите съобщения.

SPA изпраща само необходимата информация с всяко щракване, а браузърът ви я рендира. Това се различава от традиционното зареждане на страница, при което сървърът пренаписва цялата страница с всеки клик, който правите, и я изпраща на браузъра ви.



Този метод на клиентска страна по части прави времето за зареждане много по-бързо за потребителите. Той също така намалява количеството информация, което сървърът трябва да изпрати, и прави целия процес много по-ефективен от гледна точка на разходите — печеливша ситуация за потребителите и бизнесите.

** Рендиране - процесът на създаване на визуално представяне на данни или обекти, което може да бъде възприето или разбрано от потребителя.*

В. Защо да използваме SPA?

SPA имат множество предимства като част от тях са:

- **Бързо зареждане** - SPA зареждат един път и след това динамично обновяват съдържанието на страницата, което води до значително по-бързо време за зареждане в сравнение с традиционните уеб приложения с много страници.
- **Подобрено потребителско изживяване** - Заради бързото зареждане и гладката навигация без презареждане на цялата страница, потребителите получават по-добро потребителско изживяване, което води до повече удовлетворение от използването на приложението.
- **Бързо разработване на front-end** - SPA предоставят гъвкавост и улесняват разработването на front-end-а, като позволяват на разработчиците да се фокусират върху разработването на отделни компоненти и тяхната функционалност, без да се притесняват за цялостната архитектура на приложението.

- **Добри възможности за кеширане** - Благодарение на способността да кешират ресурсите си, SPA могат да подобрят производителността и бързодействието си, като намалят натоварването на



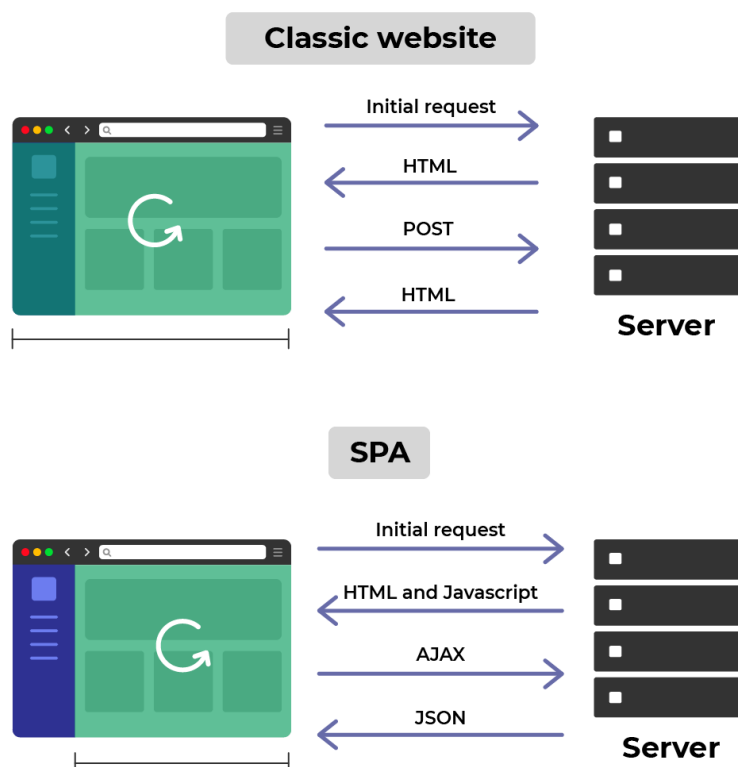
сървърите и ускорят зареждането на приложението за потребителите, което води до по-приятно изживяване за потребителите.

II. Архитектура на SPA

Multi-Page Applications (MPAs) са уеб приложения, които се състоят от множество HTML страници, като всяка страница представлява отделно заредено състояние на приложението. В MPA архитектурата, когато потребителят навигира към различни страници, се зареждат нови HTML страници от сървъра, което води до презареждане на цялата страница.

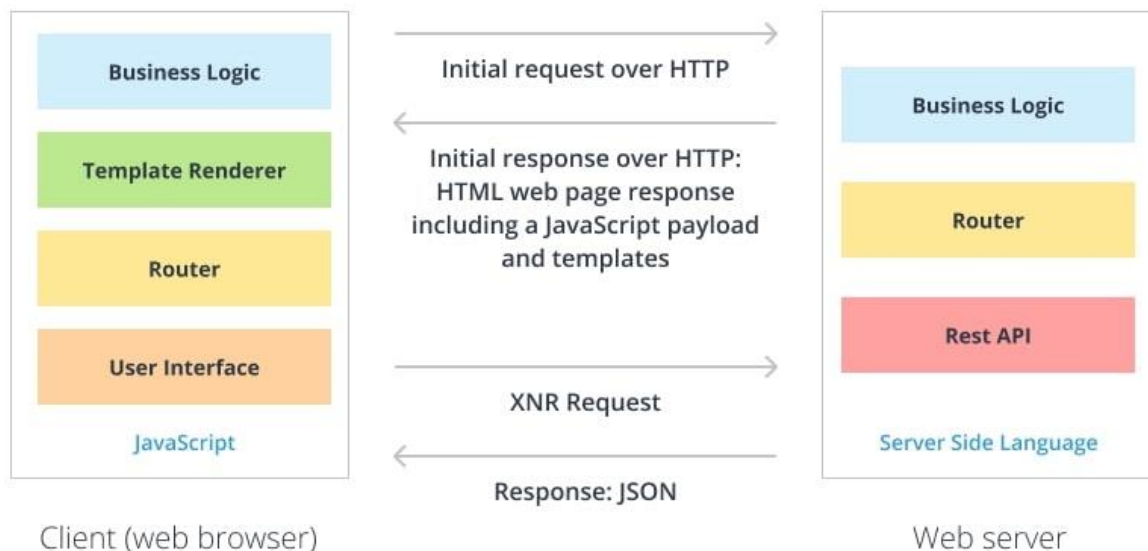
Основната разлика между MPA и SPA е в начина, по който се зарежда и рендира съдържанието. За разлика от MPA, където всеки преход към нова страница води до презареждане на цялата страница, SPA се зареждат само веднъж при първоначалното зареждане и след това динамично обновяват съдържанието на страницата, без да се налага презареждане на цялата страница.

Тази разлика прави SPA по-бързи и по-отзивчиви в сравнение с MPA, тъй като се избягва излишното презареждане на цялата страница.



Архитектурата на SPA се основава на клиент-сървър модела, където цялата бизнес логика и обработка на данни се извършват на клиентската страна, докато сървърът предоставя само необходимите данни и ресурси.

Single page app architecture diagram



Ето основните компоненти на архитектурата на SPA:

- **Клиентска част (Client-side):** Тук се намира цялата логика, която управлява визуализацията и взаимодействието на потребителите с уеб приложението. HTML съдържанието се генерира динамично с JavaScript, а CSS се прилага за стилизиране на елементите. Различни JavaScript фреймуърки и библиотеки като React, Angular или Vue.js се използват за управление на компонентите, рендерирането на страници и обработка на потребителски събития.
- **Сървърна част (Server-side):** Отговорен е за обслужването на заявките на клиента и предоставяне на необходимите данни. Сървърът може да използва различни технологии и рамки за създаване на RESTful API или GraphQL схеми за достъп до данни. Той обикновено взаимодейства с база данни или други източници на данни, обработва заявки и връща резултатите като JSON или други формати на данни.
- **Рутинг (Routing):** Рутинг системата управлява навигацията в приложението, като определя кои компоненти или страници трябва да се показват при определени URL адреси. Това позволява на потребителите да прелистват между различните секции на приложението, като се осигурява гладък и непрекъснат поток на потребителското изживяване.
- **Управление на състоянието (State Management):** Този компонент е отговорен за управлението на състоянието на приложението. Тук се съхраняват данни, които трябва да бъдат споделени между различни компоненти или състояния на приложението. Инструменти като Redux, Vuex или React Context API се използват за управление и обмен на състоянието.

- **Комуникация със сървъра (Server Communication):** Този компонент е отговорен за установяването на връзка между клиентската и сървърната част на приложението. Тук се изпращат заявки към сървъра, за да се получат необходимите данни или да се извършат операции като създаване, четене, актуализиране и изтриване на данни.

Тези компоненти работят в сътрудничество, за да създадат SPA, което е бързо, гъвкаво и лесно за употреба. Също така, те предоставят възможност за разработка на сложни функции и добавяне на нови функционалности по време на развитието на проекта.

III. Използване на HTML, CSS и Bootstrap за разработка на интерфейса на SPA

Разглеждането на HTML, CSS и Bootstrap за SPA предлага множество предимства. Те предоставят гъвкавост и лекота на използване, позволявайки бързо и лесно създаване на интерфейса. Bootstrap осигурява готови компоненти и стилове за responsive дизайн, като ускорява разработката и гарантира съвместимост с различни устройства и браузъри.

A. HTML за структуриране на страницата

- HTML (HyperText Markup Language) се използва за структуриране на съдържанието на уеб страницата, като дефинира основната си структура и елементите, които я съставят.
- За разработката на едностранично приложение (SPA), HTML играе ключова роля в дефинирането на различните компоненти и секции на приложението. Това включва съставянето на менюта, заглавия, форми, списъци и други интерактивни елементи, които потребителят може да взаимодейства.
- Всяка компонента от HTML дефинира структурата на съответната част от интерфейса и предоставя основата за визуализацията на съдържанието. Така, чрез правилно структуриране и маркиране с HTML, се осигурява ясно и лесно разбираемо представяне на информацията за потребителите.

B. CSS за стилизиране и визуално оформление

- CSS (Cascading Style Sheets) играе ключова роля в стилизирането на HTML елементите и придаването на визуално оформление на интерфейса на уеб приложението.
- Чрез CSS можем да зададем различни аспекти на дизайна, като например цветове, шрифтове, размери, отстъпи, рамки и други стилове, които съчетават външния вид и усещането на интерфейса.
- Стилите в CSS се прилагат към HTML елементите чрез селектори, които указват към кои елементи да се приложат конкретните стилове. Това позволява гъвкавост и контрол върху визуалния дизайн, като се осигурява съответствие с дизайнерските решения и потребителските очаквания.

- Правилното използване на CSS не само подобрява външния вид на интерфейса, но и подобрява потребителското изживяване, като прави навигацията по приложението по-лесна и приятна за потребителите.

What's the Difference?



HTML
Hypertext Markup Language

Create the structure

- Controls the layout of the content
- Provides structure for the web page design
- The fundamental building block of any web page



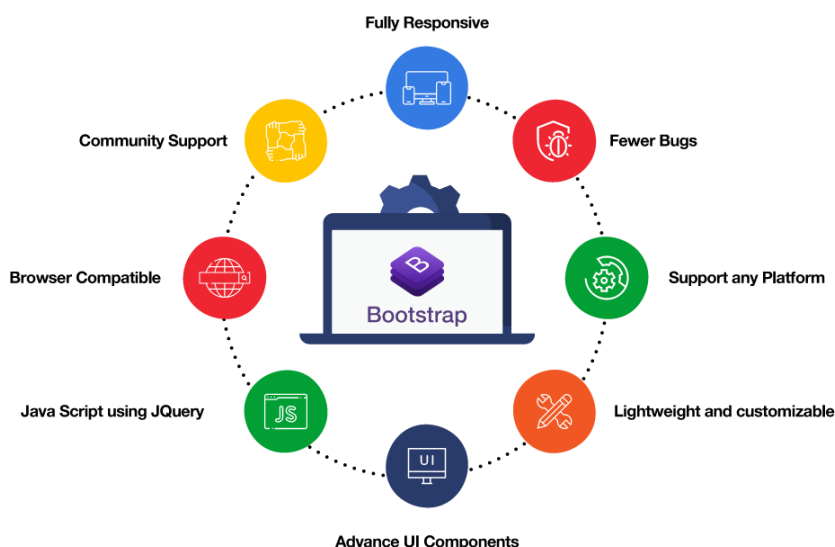
CSS
Cascading Style Sheet

Stylize the website

- Applies style to the web page elements
- Targets various screen sizes to make web pages responsive
- Primarily handles the "look and feel" of a web page

C. Bootstrap за бързо и лесно оформление на дизайна

- Bootstrap е известен CSS фреймуърк, който предлага готови компоненти, стилове и инструменти за създаване на уеб дизайн.
- С използването на Bootstrap можете лесно да добавите готови компоненти като навигационни менюта, бутони, форми, таблица, картинки и други към вашето уеб приложение.
- Една от ключовите характеристики на Bootstrap е неговата гъвкава решетка (grid), която поддържа адаптивен дизайн. Това означава, че дизайнът се адаптира към различни устройства и екрани, като например мобилни телефони, таблети и настолни компютри, без да се нарушава визуалната цялост и функционалност на приложението.
- Bootstrap предоставя множество CSS класове и компоненти, които улесняват стилизирането на уеб приложението, като осигуряват еднороден и съвременен външен вид. Това помага за ускоряване на процеса на разработка и подобрява визуалното възприятие на приложението от потребителите.





Top CSS Frameworks



Има множество други CSS framework-и, които може да разгледате, ако проявявате интерес. Линк към източник: <https://www.freecodecamp.org/news/best-css-frameworks-for-frontend-devs/>

Като цяло, използването на HTML, CSS и Bootstrap за разработка на SPA позволява на разработчиците да създадат професионално изглеждащи и функционални уеб приложения с минимални усилия и време.

IV. Принципи на SPA Frameworks

Принципът на SPA предвижда, че вече няма пълен преход на страница, както е обичайно при традиционните уеб приложения, а само промяна в потребителския интерфейс. Това позволява на приложението директно да реагира на входовете на потребителите и намалява времето за чакане за потребителите. Тъй като в SPA-та - за разлика от класическите уеб приложения - бизнес логиката, която се реализира с помощта на JavaScript, се намира на страната на клиента, сложността се увеличава на тази страна. Това го прави по-изчерпателно и трудно за разработчиците да се реализира.

За да противодействат на тази сложност и да улеснят разработката на SPA, е установено важно инструмент, наречен SPA фреймуърк. Той предоставя програмен фреймуърк, състоящ се от библиотеки, включително техните функционалности и методи.

Основните принципи на Frameworks за SPA включват:

- **Поддръжка на клиентската навигация** - SPA рамките предоставят механизми за управление на клиентската навигация, което позволява на потребителите да се движат по различните части на приложението без да е необходимо презареждане на цялата страница.

- **Съответствие** - Всички компоненти и елементи на интерфейса са съобразени с единен дизайн и стил, което осигурява консистентност и предсказуемост във визуалния опит на потребителите.
- **Кеширане на данни** - SPA рамките предоставят възможности за кеширане на данни, което позволява бърз достъп до ресурси и подобрява ефективността на приложението, като се намаляват излишните заявки към сървъра.
- **Инициализационно зареждане** - Приложенията се зареждат и инициализират един път, след което се поддържа динамично променяща се състояние без необходимост от повторно зареждане на цялата страница.
- **Зареждане на модули** - Рамките за едностранични приложения позволяват зареждането на различни модули и компоненти по изискване, което подобрява скоростта на зареждане на приложението и управлението на ресурсите.

Тези принципи осигуряват ефективно и гъвкаво разработване на SPA, като подобряват потребителския опит и управлението на приложението в цялост.

V. Frameworks за SPA

В постоянно променящия се пейзаж на уеб разработката, JavaScript е езикът, който играе критична роля в създаването на динамични и интерактивни уеб приложения. JavaScript се използва за добавяне на функционалност към уеб страници, като например валидация на форми, анимации, взаимодействие с потребителя и обновяване на съдържанието на страницата без необходимост от презареждане.

За да се създадат SPAs ефективно, JavaScript е от съществено значение, тъй като езикът позволява динамичното обновяване на съдържанието на страницата в реално време, в зависимост от действията на потребителя. SPAs се отличават с това, че зареждат една HTML страница и след това използват

Principles of SPA frameworks

01.	Client navigation support	Every user's transition to and from module pages must be explicitly fixed into the navigation history. The navigation must be "deep" , which means that if the user will copy and paste the link of an internal module page in another browser or window, they will end up on the copied page.
02.	Consistency	Every bit of data (scripts, styles) required for the application work must be defined on a single page .
03.	Caching storage	The working state of the client must be continuously stored in the browser cache or in the WebStorage .
04.	Initialization load	All scripts required for the application start must be loaded at initialization .
05.	Module load	Modules subsequent to the main ones must be loaded on demand .



JavaScript

Increase interactivity

- Adds interactivity to a web page
- Handles complex functions and features
- Programmatic code which enhances functionality

JavaScript за динамично обновяване на съдържанието, без да е необходимо презареждане на цялата страница. Това предоставя потребителите с безпроблемно и непрекъснато потребителско изживяване, като минимизира времето за зареждане и подобрява ефективността на приложението.

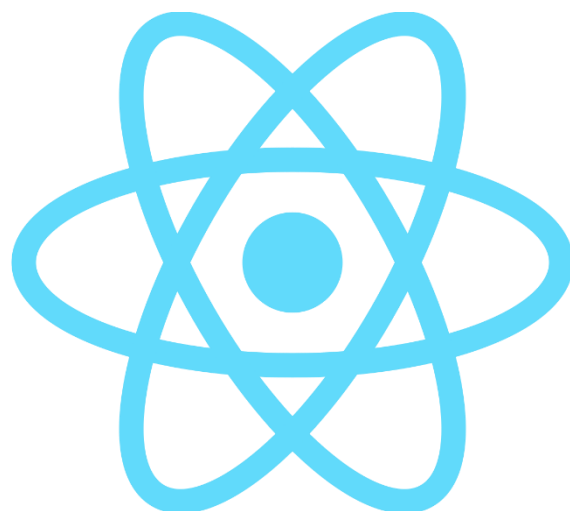
1. React

React, разработен от Facebook, е една от най-популярните и широко приети JavaScript библиотеки за създаване на SPAs. Неговата компонентна базирана архитектура, виртуален DOM и силната подкрепа от общността го правят отличен избор. Гъвкавостта на React позволява на разработчиците да го интегрират с други библиотеки и рамки, като ви дава свобода да избирате най-добрите инструменти за вашия проект.

Източници:

<https://react.dev/>

<https://www.w3schools.com/REACT/DEFAULT.ASP>



2. Angular

Angular е рамка на Google, която предоставя структуриран подход за създаване на SPAs. Той предлага функции като двустранно свързване на данни, инжектиране на зависимости и мощен CLI инструмент за създаване на проекти. Обширната му документация и голямата общност го правят надежден избор за сложни приложения.

Източници:

<https://angular.io/tutorial>

<https://www.w3schools.com/angular/>



3. Vue.js

Vue.js е прогресивна JavaScript рамка, която позволява на разработчиците постепенно да въвеждат неговите функции. Той предлага прост и достъпен API, като го прави отличен избор за начинаещи и опитни разработчици. Екосистемата на Vue включва Vue Router за маршрутизация и Vuex за управление на състоянието, което гарантира гладко изживяване при разработването на SPAs.

Източници:

<https://vuejs.org/tutorial/#step-1>

<https://www.w3schools.com/vue/>





ember

4. Ember.js

Ember.js е рамка, която налага конвенции пред конфигурации. Той разполага с набор от вградени инструменти и практики, което го прави лесно да се създадат амбициозни SPAs. Инструментът "ember-cli" на Ember предоставя генератори за общите задачи, а силната му насоченост към разработчиците води до увеличена производителност.

Източници:

<https://guides.emberjs.com/release/tutorial/part-1/>

<https://www.tutorialspoint.com/emberjs/index.htm>

5. Svelte

Svelte представя различен подход за създаване на SPAs, като компилира компоненти в ефективен JavaScript по време на създаване на проект. Това води до изключително ефективни приложения с по-малки размери на пакетите. Простотата на Svelte и неговото внимание към реактивността го правят интригуващ избор за разработчиците, които търсят алтернатива на традиционните рамки.



Източници:

<https://learn.svelte.dev/tutorial/welcome-to-svelte>



6. Aurelia

Aurelia е рамка, която насочва към простота и конвенции пред конфигурации. Той предлага двустранно свързване на данни, мощна система за маршрутизация и плъгинова архитектура. Чистият и модулен код на Aurelia го прави отличен избор за създаване на поддържаеми SPAs.

Източници:

<https://aurelia.io/docs/tutorials/>

<https://www.tutorialspoint.com/aurelia/index.htm>

7. Backbone.js

Backbone.js е лека рамка, която предоставя основната структура за създаване на SPAs. Той предлага модели, изгледи, колекции и маршрутизатори, позволявайки на разработчиците да избират допълнителни библиотеки и инструменти, за да допълнят техните приложения. Простотата и гъвкавостта на Backbone го правят подходящ за малки до средни проекти.

Източници:

<https://backbonejs.org/>

<https://www.tutorialspoint.com/backbonejs/index.htm>



8. Mithril

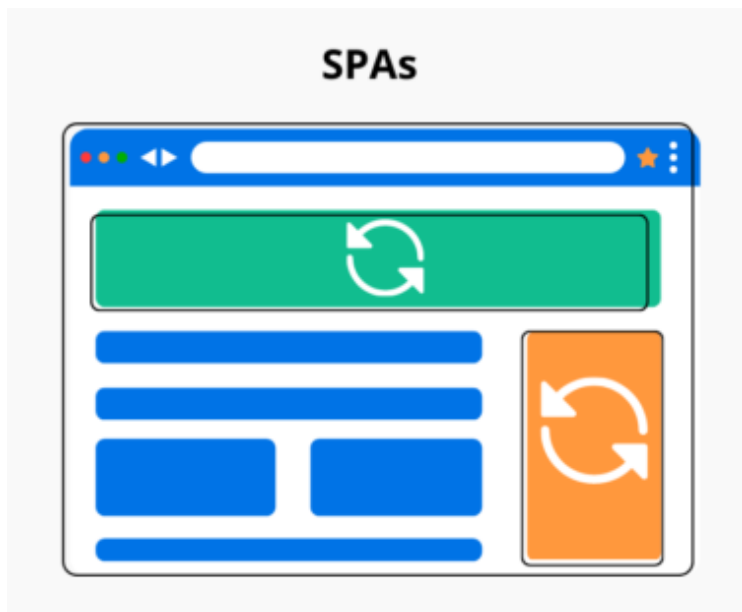
Mithril е минималистична JavaScript рамка, предназначена за създаване на бързи и малки SPAs. Той се отличава с малък размер на пакета и предлага конкретно API за създаване на компоненти и управление на състоянието на приложението. Подходът на Mithril към производителността го прави идеален за проекти, където оптимизацията е от първостепенно значение.

Източници:

<https://mithril.js.org/>

Изборът на правилния рамка за SPA е от съществено значение за успеха на вашия проект за уеб разработка. Всяка от рамките, споменати по-горе, имат своите силни страни и слабости, които се отнасят до различни предпочитания на разработчиците и изисквания на проекта. Обмислете вашите конкретни нужди, експертиза на екипа и мащаб на проекта, когато вземате вашето решение. Най-важното е, че най-добрият рамка за SPA е този, който ви дава възможност да създадете безпроблемно и ефективно потребителско изживяване, докато се съобразявате с вашите цели за разработка.

VI. Стъпки за реализация на SPA с използване на HTML, CSS и Bootstrap



A. Създаване на основната HTML структура

- Началната стъпка е създаването на основната HTML структура на вашето SPA. Тук трябва да включите основни елементи като заглавие, навигационно меню, основно съдържание и footer.
- Разгледайте кой ще бъде контейнерният елемент, където ще се зареждат различните компоненти на вашето SPA. Това обикновено е единственият елемент в HTML, който ще се променя динамично при навигацията в приложението.

B. Добавяне на стилове с CSS

- След като сте създали HTML структурата, добавете стилове с CSS, за да оформите интерфейса на вашето SPA. Тук можете да използвате различни CSS техники за да стилизирате елементите, като цветове, шрифтове, размери, отстъпи и т.н.

C. Използване на Bootstrap за създаване на респонсивен дизайн

- Включете Bootstrap в проекта си и използвайте неговите класове и компоненти за създаване на респонсивен дизайн. Bootstrap предоставя готови компоненти и стилове, които могат да бъдат лесно интегрирани във вашето SPA, като по този начин улесняват създаването на атрактивен интерфейс.

D. Интегриране на фреймуърк за SPA

- Включете фреймуърк за SPA като React или Vue.js в проекта си. Тези инструменти позволяват създаването на динамични и интерактивни компоненти на вашето SPA.

Е. Създаване на компоненти и управление на състоянието

- Създайте различни компоненти за вашето SPA, като разделяте интерфейса на части за повторно използване. Използвайте механизми за управление на състоянието, като например context в React или Vuex в Vue.js, за да съхранявате и обменяте данни между компонентите.

Ф. Добавяне на маршрутизация и навигация

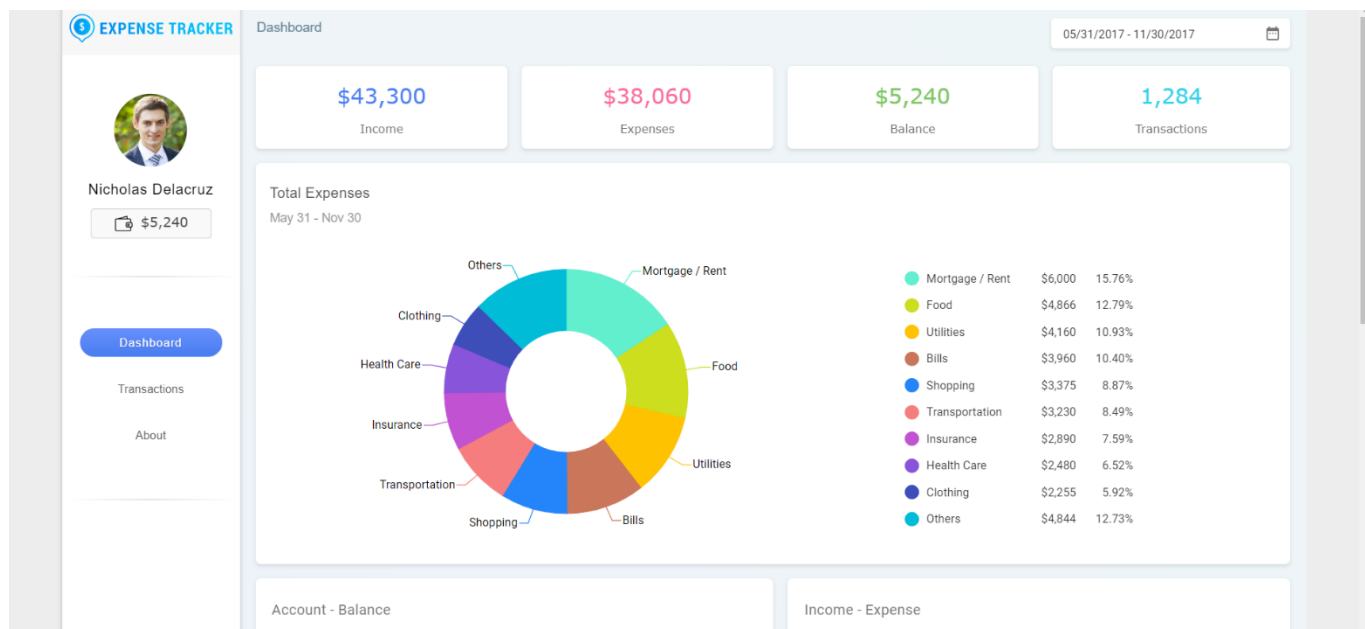
- Интегрирайте маршрутизация във вашето SPA, за да позволите на потребителите да навигират между различните страници или секции на приложението без да се налага презареждане на цялата страница. Използвайте библиотеки като react-router в React или vue-router в Vue.js за управление на маршрутите и навигацията.

Следвайки тези стъпки, ще можете да създадете функционално и атрактивно SPA с използване на HTML, CSS и Bootstrap, което е готово за разширение и развитие.

VII. Примери за SPA

Програма за проследяване на разходите:

- Това приложение демонстрира различни контроли на Essential JS 2, които се използват за проследяване и визуализиране на разходите с отличен потребителски интерфейс.
- Потребителите могат да навигират между различните уебстраници чрез URL адресите, без да е необходимо презареждане на страницата.
- Това приложение се отличава с responsive дизайн, който позволява използването му както на десктопни браузъри, така и на мобилни устройства.
- Линк към страницата -> https://ej2.syncfusion.com/showcase/typescript/expensetracker/?_gl=1*1l747s*_ga*MzY5NDExOTgxLjE3MTMyOTA0NzA.*_ga_41J4HFMX1J*MTcxMzI5MDQ3MC4xLjAuMTcxMzI5MDQ3MC4wLjAuMA..#/dashboard
- Линк към GitHub -> <https://github.com/syncfusion/ej2-showcase-js-expensetracker/blob/master/src/dashboard/dashboard.js>



Проследяване на здравето

- Health Tracker е още един пример за SPA, което проследява здравето на човек, като записва приема на храна, изминатото разстояние, консумацията на вода и продължителността на съня.
- Това приложение е създадено с богат интерфейс и интерактивен дизайн, използвайки контроли на Syncfusion като Графики и Картички като уебстраница с единичен изглед.
- Линк към страницата -> https://ej2.syncfusion.com/showcase/typescript/healthtracker/?_gl=1*16ark9t*_ga*MzY5NDEExOTgxLjE3MTMyOTA0NzA.*_ga_41J4HFMX1J*MTcxMzI5MDQ3MC4xLjEuMTcxMzI5MDcxOC4wLjAuMA..
- Линк към GitHub -> <https://github.com/syncfusion/ej2-showcase-ts-healthtracker>



Проектиране на потребителски интерфейси

VIII. Предизвикателства на SPA

Въпреки че SPA имат своите предимства, общите проблеми, включват:

- **Затруднения при мащабирането.** Увеличен трафик или затруднения при зареждането на уебсайта може да възникнат, тъй като SPA често остават зависими от ограничен брой сървъри за зареждане и поддържане на потребителското изживяване.
- **Рискове от сигурността.** С голямата част от работата, извършвана от страна на клиента, SPA продължават да бъдат податливи на атаки, които могат да внедрят злонамерен код, който се изпълнява в браузъра. Тези атаки могат да бъдат предотвратени, следвайки подходящи мерки за сигурност, включително Политика за сигурност на съдържанието (CSP).
- **Недостатъци в SEO.** Тъй като част от архитектурата на SPA се изобразява само една страница, те остават трудни за оптимизация за търсещите машини. Въпреки това, Google е започнал да предоставя по-добри инструменти за индексване на SPA.
- **Анализ на сайта.** Този процес изисква активен подход, тъй като се изобразява само една страница, което прави по-трудно определянето на трафика към уебсайта или къде потребителите отпадат в клиентския път.
- **Предизвикателства при навигацията.** Съдържанието трябва да бъде проектирано така, че да съответства на съществуващото потребителско изживяване при всяко ново представяне на информация, за да се елиминира неудобното превъртане на страницата. Тези проблеми се отнасят и до навигацията, тъй като бутона "Назад" не функционира по същия начин в SPA.
- **JavaScript.** За потребителите, които имат деактивиран JavaScript, независимо дали по избор или от техния корпоративен IT екип, SPA просто няма да работят. JavaScript е необходим за изобразяване на потребителския интерфейс, от който зависи SPA, което прави това проблематично в някои случаи.
- **Проблеми с паметта.** Някои рамки могат да причинят заделянето на памет, когато тя вече не е необходима, което може да засегне производителността на приложението.

